

TÀI LIỆU HƯỚNG DẪN

MOCK PROJECT C++

MÔ PHỎNG PHẦN MỀM NHÚNG CƠ BẢN

Thời gian thực hiện: 2 tuần | Quy mô: 5–6 sinh viên/nhóm

Nội dung	Quy định
Ngôn ngữ	C++
Hình thức	Ứng dụng Console trên máy tính
Phần cứng	Không sử dụng phần cứng, vi điều khiển hay cảm biến thật
Trọng tâm	Sensor/Event giả lập → Controller → Actuator giả lập → Display/Logger
Thời lượng	10 ngày làm việc
Sản phẩm	Source code + tài liệu thiết kế + test case + log + demo/defense

1. Mục tiêu của Mock Project

Mock Project giúp sinh viên vận dụng C++ vào một bài toán có tư duy gần với phần mềm nhúng nhưng được thực hiện hoàn toàn trên máy tính. Các thiết bị vật lý được thay thế bằng dữ liệu, sự kiện và trạng thái giả lập.

- Hiểu mô hình Input/Sensor → Controller → Output/Actuator.
- Thực hành C++ cơ bản, hàm, class, enum, array/vector và chia module.
- Hiểu state, event, transition và vòng lặp điều khiển.
- Thực hành timing bằng `std::chrono` và `std::this_thread::sleep_for()`.
- Xử lý dữ liệu bất thường và trạng thái lỗi.
- Ghi log hoạt động bằng `fstream`.
- Thực hành Git, tích hợp code, testing và trình bày kỹ thuật.

2. Kiến trúc chung bắt buộc

```
SENSOR / EVENT GIẢ LẬP
  |
  v
CONTROLLER
  |
  v
ACTUATOR / OUTPUT GIẢ LẬP
  |
  v
DISPLAY + LOGGER
```

Sinh viên không nên viết toàn bộ chương trình trong `main()`. Mỗi thành phần cần có trách nhiệm rõ ràng. Hàm `main()` chủ yếu khởi tạo hệ thống và điều phối vòng lặp.

```
while (systemRunning) {
    readInput();
    processLogic();
    updateOutput();
    displayStatus();
    writeLog();
    waitNextCycle();
}
```

3. Yêu cầu kỹ thuật chung

Hạng mục	Yêu cầu
C++	Bắt buộc
Console	Bắt buộc; GUI không yêu cầu
Hardware	Không sử dụng
Sensor/Input	Sinh dữ liệu bằng chương trình hoặc nhập từ bàn phím
Controller	Tách riêng logic quyết định
State	Dùng enum/enum class khi phù hợp
Timing	<code>chrono/thread</code> ở mức cơ bản
Data Structure	array/vector cơ bản
File	Ghi <code>system.log</code> bằng <code>fstream</code>
Error handling	Ít nhất 2 tình huống bất thường
Source	Tách <code>.h</code> và <code>.cpp</code>
Git	Mọi thành viên phải có commit có ý nghĩa
Không yêu cầu	Database, network, Web API, external library

4. Nhóm 1 – Temperature Control System Simulator

4.1. Bối cảnh

Xây dựng chương trình mô phỏng hệ thống giám sát nhiệt độ phòng máy. Cảm biến nhiệt độ được giả lập bằng phần mềm. Controller đọc nhiệt độ và quyết định trạng thái của quạt và cảnh báo.

TemperatureSensor → TemperatureController → Fan / Alarm → Display / Logger

4.2. Luồng hoạt động

```
Đọc nhiệt độ
↓
Kiểm tra dữ liệu hợp lệ?
  ↓ Có          ↓ Không
Phân loại mức    Báo SENSOR ERROR
↓
Điều khiển Fan/Alarm
↓
Hiển thị + ghi log
↓
Chờ chu kỳ tiếp theo
```

4.3. Yêu cầu chức năng

ID	Yêu cầu
TC-01	Cho phép AUTO: chương trình tự sinh nhiệt độ; MANUAL: người dùng nhập nhiệt độ.
TC-02	Quy định khoảng nhiệt độ hợp lệ, ví dụ 0–60°C.
TC-03	Nhiệt độ < 30°C: Fan OFF, Alarm OFF.
TC-04	Nhiệt độ 30–39°C: Fan ON, Alarm OFF.
TC-05	Nhiệt độ >= 40°C: Fan ON, Alarm ON và hiển thị cảnh báo.
TC-06	Hiển thị Temperature, Fan, Alarm và trạng thái hệ thống.
TC-07	Hệ thống chạy theo chu kỳ, ví dụ 1 giây/lần.
TC-08	Dữ liệu sensor không hợp lệ phải được phát hiện và không xử lý như dữ liệu bình thường.
TC-09	Ghi giá trị nhiệt độ, thay đổi trạng thái và lỗi vào system.log.

4.4. Class/module gợi ý

```
TemperatureSensor
TemperatureController
Fan
Alarm
Display
Logger
```

4.5. Kết quả hiển thị mẫu

```
===== TEMPERATURE CONTROL =====
Temperature : 37 C
Fan         : ON
Alarm       : OFF
System      : RUNNING
=====
```

4.6. Test case tối thiểu

Test	Điều kiện	Kết quả mong đợi
T1	25°C	Fan OFF, Alarm OFF
T2	35°C	Fan ON, Alarm OFF
T3	45°C	Fan ON, Alarm ON
T4	Giá trị sensor lỗi	Thông báo lỗi; không điều khiển dựa trên dữ liệu lỗi

T5	45°C → 28°C	Fan/Alarm trở về trạng thái phù hợp
----	-------------	-------------------------------------

4.7. Nâng cao không bắt buộc

- Cho phép cấu hình threshold.
- Thêm trạng thái NORMAL/WARNING/CRITICAL.
- Tính min/max/average temperature.
- Random sensor failure.

5. Nhóm 2 – Smart Traffic Light Controller Simulator

5.1. Bối cảnh

Xây dựng bộ điều khiển đèn giao thông giả lập bằng C++. Project tập trung vào Finite State Machine (FSM), timer và event. Không sử dụng LED hay nút bấm thật.

```
RED → GREEN → YELLOW → RED → ...
```

5.2. Luồng cơ bản

```
Current State + Timer + Event
    ↓
Controller
    ↓
Next State
    ↓
Display Vehicle/Pedestrian Light
    ↓
Log
```

5.3. Yêu cầu chức năng

ID	Yêu cầu
TL-01	Có tối thiểu RED, GREEN và YELLOW cho xe.
TL-02	Tự chuyển RED → GREEN → YELLOW → RED.
TL-03	Có countdown timer cho state hiện tại.
TL-04	Giả lập nút P để tạo yêu cầu người đi bộ.
TL-05	Yêu cầu người đi bộ phải được xử lý qua chuyển trạng thái an toàn.
TL-06	Hiển thị vehicle light, pedestrian light, state và remaining time.
TL-07	Tuyệt đối không để Vehicle GREEN và Pedestrian GREEN cùng lúc.
TL-08	Có Emergency Mode đưa hệ thống về trạng thái an toàn đã định nghĩa.
TL-09	Ghi log state transition, pedestrian request và emergency event.

5.4. State gợi ý

```
enum class TrafficState {
    RED,
    GREEN,
    YELLOW,
    PEDESTRIAN,
    EMERGENCY
};
```

Nhóm bắt buộc phải vẽ State Diagram trước khi hoàn thiện code Controller.

5.5. Console mẫu

```
===== TRAFFIC CONTROLLER =====
Vehicle Light      : GREEN
Pedestrian Light   : RED
Remaining Time     : 7 s
Pedestrian Req.    : NO
=====
P = Pedestrian request | E = Emergency
```

5.6. Test case tối thiểu

Test	Điều kiện	Kết quả mong đợi
------	-----------	------------------

T1	Normal mode	RED → GREEN → YELLOW → RED đúng thứ tự
T2	Nhấn P khi GREEN	Ghi nhận request và chuyển qua luồng an toàn
T3	Pedestrian GREEN	Vehicle bắt buộc RED
T4	Emergency	Chuyển về safe state
T5	Thoát Emergency	Quay lại normal mode theo quy tắc đã thiết kế

5.7. Nâng cao không bắt buộc

- Cấu hình thời gian từng state.
- Mô phỏng hai hướng đường.
- Traffic density giả lập.
- Countdown cho người đi bộ.

6. Nhóm 3 – Smart Parking Controller Simulator

6.1. Bối cảnh

Xây dựng chương trình mô phỏng controller của bãi đỗ xe thông minh. Entry/Exit Sensor, barrier và parking slot đều là đối tượng phần mềm, không phải thiết bị thật.

```
Entry/Exit Event → ParkingController → Barrier + ParkingSlot + Display + Logger
```

6.2. Luồng xe vào

```
Car detected at ENTRY
↓
Kiểm tra chỗ trống
  ↓ Có      ↓ Không
Open Barrier  PARKING FULL
                ↓
                Barrier CLOSED
↓
Xe vào
↓
Occupy Slot
↓
Close Barrier
↓
Update Display + Log
```

6.3. Yêu cầu chức năng

ID	Yêu cầu
PK-01	Khởi tạo số lượng slot, ví dụ 10.
PK-02	Mỗi slot có trạng thái FREE hoặc OCCUPIED.
PK-03	Giải lập event xe đến cổng vào.
PK-04	Controller phải kiểm tra capacity trước khi mở barrier.
PK-05	Nếu còn chỗ: mở barrier → xe vào → occupy slot → đóng barrier.
PK-06	Nếu đầy: từ chối xe, barrier giữ CLOSED và báo PARKING FULL.
PK-07	Giải lập xe ra; giải phóng một slot và cập nhật capacity.
PK-08	Barrier tối thiểu có OPEN/CLOSED.
PK-09	Hiển thị Total, Occupied, Available và trạng thái barrier/slot.
PK-10	Không cho phép Occupied > Total hoặc Available < 0.
PK-11	Xử lý event bất thường: xe ra khi bãi trống, input/sensor không hợp lệ.
PK-12	Ghi entry, exit, barrier, slot và error event vào log.

6.4. Class/module gợi ý

```
ParkingSlot
EntrySensor
ExitSensor
Barrier
ParkingController
Display
Logger
```

6.5. Console mẫu

```
===== SMART PARKING =====
Total Slots : 10
Occupied    : 7
Available   : 3

P01 [X] P02 [ ] P03 [X] P04 [X] P05 [ ]
P06 [X] P07 [X] P08 [ ] P09 [X] P10 [X]
```

```

Entry Barrier : CLOSED
Exit Barrier  : CLOSED
=====
1. Simulate car entering
2. Simulate car leaving
3. Show status
0. Exit

```

6.6. Test case tối thiểu

Test	Điều kiện	Kết quả mong đợi
T1	Xe vào khi còn chỗ	Barrier mở; một slot chuyển OCCUPIED
T2	Xe vào khi đầy	Từ chối; barrier không mở
T3	Xe ra	Một slot chuyển FREE
T4	Xe ra khi bãi trống	Xử lý lỗi an toàn
T5	Nhiều lần vào/ra	Counter và trạng thái slot luôn nhất quán

6.7. Nâng cao không bắt buộc

- Gán Vehicle ID cho slot.
- Barrier: OPENING/OPEN/CLOSING/CLOSED.
- Tính thời gian đỗ xe.
- Waiting queue khi bãi đầy.

7. Kế hoạch thực hiện trong 2 tuần

Ngày	Công việc	Đầu ra
1	Đọc đề, phân tích requirement, tạo Git repository	Requirement list + task board
2	Thiết kế kiến trúc, class và state	Block/Class/State Diagram
3	Tạo project skeleton, header/source	Project compile được
4	Xây dựng Sensor/Input simulation	Input hoạt động
5	Xây dựng Controller logic	Core rules hoạt động
6	Xây dựng Actuator/Output/State	Output hoạt động
7	Timing, FSM/Event và Logging	Control loop tích hợp
8	Integration + abnormal handling	Feature complete
9	Testing, sửa lỗi, README	Test evidence + release candidate
10	Demo, code review, defense	Final submission

8. Gợi ý phân công nhóm 5–6 sinh viên

Vai trò	Trách nhiệm chính
Team Leader	Theo dõi requirement, task, merge/integration
Input/Sensor	Mô phỏng sensor/event và validate input
Controller	Xử lý logic điều khiển
Output/State	Actuator/state/display
Timing/Logger	Timer, logging, error handling
Tester/Document	Test case, tài liệu, hỗ trợ integration

Phân công chỉ xác định trách nhiệm chính. Tất cả thành viên phải hiểu luồng tổng thể và có thể bị hỏi bất kỳ module nào trong buổi defense.

9. Cấu trúc source và sản phẩm nộp

```
Group_X_Project/  
├── include/  
│   └── *.h  
├── src/  
│   └── *.cpp  
├── logs/  
│   └── system.log  
├── docs/  
│   ├── Requirement.md  
│   ├── Block_Diagram.png  
│   ├── State_Diagram.png  
│   └── Test_Cases.xlsx  
├── main.cpp  
└── README.md
```

- Source code compile và chạy được.
- README hướng dẫn build/run và phân công thành viên.
- Requirement, Block Diagram và State Diagram.
- Test Cases kèm kết quả test.
- File log sinh ra từ chương trình.
- Git history thể hiện đóng góp thực tế của từng thành viên.

10. Quy tắc coding

- Không viết toàn bộ logic trong main.cpp.
- Đặt tên class/function/variable có ý nghĩa.
- Tách Input/Sensor khỏi Controller và Output.

- Không lặp lại logic điều khiển ở nhiều nơi.
- Threshold/duration nên dùng constant/config thay vì magic number.
- Validate input trước khi xử lý.
- Chương trình không được crash trong các test case bắt buộc.
- Không cố đưa inheritance/polymorphism nếu bài toán không cần; ưu tiên thiết kế rõ ràng.

11. Tiêu chí đánh giá đề xuất

Tiêu chí	Trọng số	Nội dung
Đúng chức năng	30%	Các requirement bắt buộc hoạt động đúng
C++ & cấu trúc code	20%	Class, function, module, enum, data structure
Tư duy Embedded Software	15%	Input→Controller→Output, state/event/timing
Testing & Robustness	15%	Normal, boundary, abnormal cases
Teamwork & Git	10%	Contribution và integration
Tài liệu & Defense	10%	Diagram, README, giải thích và demo

12. Yêu cầu Demo & Defense

- Giới thiệu bài toán và kiến trúc hệ thống.
- Demo một normal flow hoàn chỉnh.
- Demo state/event transition.
- Demo ít nhất 2 boundary/abnormal cases.
- Mở file log và giải thích các event.
- Giải thích Controller quyết định output/next state như thế nào.
- Giải thích vai trò các class/module chính.
- Mỗi thành viên phải trả lời câu hỏi code review.

Lưu ý: Project không được đánh giá chỉ dựa trên việc console hiển thị đúng. Nhóm phải chứng minh được thiết kế hợp lý, logic xử lý đúng và hiểu được source code.

13. Giới hạn phạm vi

Ưu tiên hoàn thành chắc chắn các yêu cầu bắt buộc. Không cần GUI, database, network, Web API, Arduino, ESP32, cảm biến hay thiết bị vật lý. Chỉ thực hiện phần nâng cao sau khi mandatory requirements và test cases đã hoàn thành.